



KONGU ENGINEERING COLLEGE

(Autonomous)

Perundurai, Erode – 638 060



DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

Name : SIVA K
Register Number : 24MCR101
Branch : Computer Applications
Semester : III

Certified that this is a bonafide record of work done by the above student for
24MCF07 – DEEP LEARNING LABORATORY during the academic year
2025- 2026.

Submitted for the End Semester Examination held on _____

Course-in-Charge

Head of the Department

Examiner -1

Examiner -2

INDEX

EX.NO	DATE	NAME OF THE EXPERIMENTS	PAGE NO.	MARKS	SIGNATURE
1.		Simple perceptron			
2.		Multilayer perceptron			
3.		Traditional data augmentation			
4.		Image Data generator class in data augmentation			
5.		CNN process for Image classification			
6.		RNN architecture for time series data			
7.		Text analysis using NLP			
8.		Deep Dream and NST			
9.		Synthetic image using VAE			
10.		Synthetic image using GAN			

EX. NO: 04

DATE :

IMAGE DATA GENERATOR FOR IMAGE DATA AUGMENTATION

AIM:

To demonstrate the use of the Keras ImageDataGenerator class for performing real-time image data augmentation in order to increase dataset diversity and improve model generalization.

ALGORITHM:

STEP 1: Install required libraries (numpy, matplotlib, tensorflow).

STEP 2: Import Packages From tensorflow.keras.preprocessing.image, import ImageDataGenerator, load_img, and img_to_array. Import matplotlib.pyplot for displaying images.

STEP 3: Create an ImageDataGenerator object with augmentation parameters such as rotation_range, width_shift_range, height_shift_range, shear_range, zoom_range, horizontal_flip, and fill_mode.

STEP 4: Load a sample image using load_img(). Convert it to a NumPy array using img_to_array() and reshape it to (1, height, width, channels) for batch processing.

STEP 5: Use datagen.flow() to create an iterator that generates augmented image batches in real time.

STEP 6: Visualize Augmentations Iterate through the batches, display several augmented images using matplotlib.pyplot.imshow().

STEP 7: Use flow_from_directory() or flow() with a training directory or dataset to supply augmented images directly to a deep learning model during the model.fit() training process

STEP 8: Train the model using the augmented data and evaluate its performance on a validation or test set to observe improved generalization and reduced overfitting.

PROGRAM:

```
import gdown

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import ImageDataGenerator, img_to_array,
load_img

file_id = "1q3tzA8L0rpZvwQG_5MxsjvhZi5q6DCJH"

gdown.download(f"https://drive.google.com/uc?id={file_id}", "simba.jpg", quiet=False)

img_path = "simba.jpg"

image = load_img(img_path, target_size=(224, 224))

image_array = img_to_array(image)

image_array = np.expand_dims(image_array, axis=0)

plt.figure(figsize=(5, 5))

plt.imshow(image_array[0].astype("uint8"))

plt.title("Original Image")

plt.axis("off")

plt.show()

datagen = ImageDataGenerator(
    rotation_range=40,
    zoom_range=0.2,
    brightness_range=[0.5, 1.5],
    shear_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

plt.figure(figsize=(10, 10))

i = 0

for batch in datagen.flow(image_array, batch_size=1):
    plt.subplot(3, 3, i + 1)
    plt.imshow(batch[0].astype("uint8"))
```

```
plt.axis("off")  
i += 1  
if i >= 9:  
    break  
plt.suptitle("Augmented Images")  
plt.show()
```

OUTPUT:



Augmented Images



COE (20)	
RECORD (20)	
VIVA (10)	
TOTAL (50)	

RESULT:

The image was successfully downloaded, processed, and augmented. Multiple transformed versions including rotations, zooms, brightness changes, shears, and horizontal flips were generated and displayed, demonstrating effective real-time image data augmentation using Keras ImageDataGenerator.

EX. NO: 05

DATE :

CNN MODEL FOR IMAGE CLASSIFICATION

AIM:

To build, train, and evaluate a Convolutional Neural Network (CNN) to classify images into ten categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck) using the CIFAR-10 dataset.

ALGORITHM:

STEP 1: Import required libraries: tensorflow, matplotlib, and required Keras modules (datasets, layers, models).

STEP 2: Load the CIFAR-10 dataset and split into training and testing sets.

STEP 3: Normalize pixel values of images to the range $[0, 1]$ for faster convergence.

STEP 4: Build the CNN by stacking Conv2D and MaxPooling2D layers for feature extraction, then flatten the output, add a Dense layer with ReLU for non-linear learning, and finish with a 10-unit Dense layer for class prediction.

STEP 5: Compile the model using the Adam optimizer, SparseCategoricalCrossentropy loss, and accuracy as the metric.

STEP 6: Train the model for 10 epochs with the training set and validate on the test set.

STEP 7: Evaluate the trained model on the test data and print test accuracy.

STEP 8: Plot the training and validation accuracy curves to visualize performance.

PROGRAM:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
```

```

train_images, test_images = train_images / 255.0, test_images / 255.0
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10) # 10 classes for CIFAR-10
])
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training and Validation Accuracy')
plt.grid(True)
plt.show()

```

OUTPUT:

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>

170498071/170498071 ————— 4s 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113:

UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Epoch 1/10

1563/1563 ————— 67s 42ms/step - accuracy: 0.3552
- loss: 1.7377 - val_accuracy: 0.5393 - val_loss: 1.2665

Epoch 2/10

1563/1563 ————— 76s 38ms/step - accuracy: 0.5684
- loss: 1.2048 - val_accuracy: 0.6026 - val_loss: 1.1164

Epoch 3/10

1563/1563 ————— 84s 40ms/step - accuracy: 0.6377
- loss: 1.0284 - val_accuracy: 0.6516 - val_loss: 0.9907

Epoch 4/10

1563/1563 ————— 61s 39ms/step - accuracy: 0.6727
- loss: 0.9251 - val_accuracy: 0.6660 - val_loss: 0.9627

Epoch 5/10

1563/1563 ————— 83s 40ms/step - accuracy: 0.7040
- loss: 0.8493 - val_accuracy: 0.6810 - val_loss: 0.9222

Epoch 6/10

1563/1563 ————— 83s 40ms/step - accuracy: 0.7249
- loss: 0.7879 - val_accuracy: 0.6711 - val_loss: 0.9427

Epoch 7/10

1563/1563 ————— 82s 40ms/step - accuracy: 0.7422
- loss: 0.7412 - val_accuracy: 0.6942 - val_loss: 0.9027

Epoch 8/10

1563/1563 ————— 82s 40ms/step - accuracy: 0.7600
- loss: 0.6888 - val_accuracy: 0.7103 - val_loss: 0.8535

Epoch 9/10

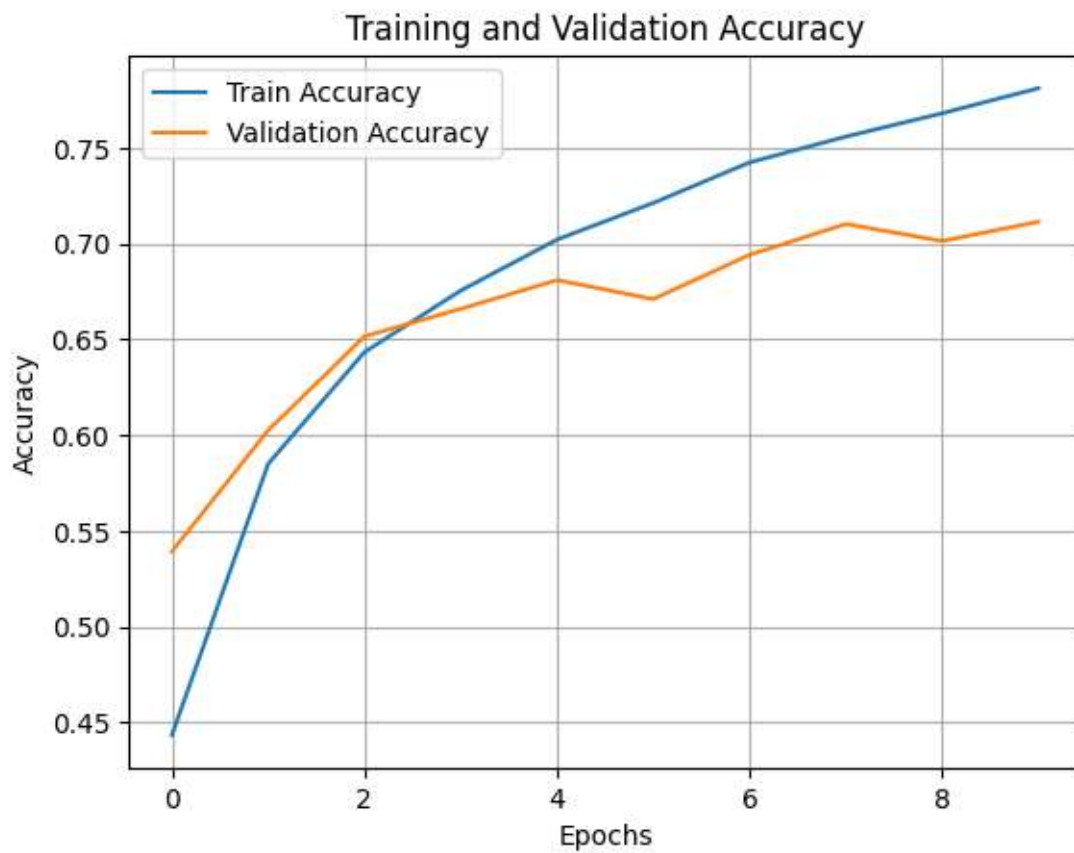
1563/1563 ————— 62s 39ms/step - accuracy: 0.7774
- loss: 0.6355 - val_accuracy: 0.7014 - val_loss: 0.8914

Epoch 10/10

1563/1563 ————— 81s 39ms/step - accuracy: 0.7893
- loss: 0.6075 - val_accuracy: 0.7115 - val_loss: 0.8591

313/313 - 4s - 14ms/step - accuracy: 0.7115 - loss: 0.8591

Test Accuracy: 71.15%



COE (20)	
RECORD (20)	
VIVA (10)	
TOTAL (50)	

RESULT:

The CNN was successfully trained and tested on the CIFAR-10 dataset. It accurately classified images into their respective categories, achieving high test accuracy and demonstrating effective feature extraction for image classification.

EX. NO: 06

DATE :

RNN ARCHITECTURE FOR TIME SERIES PREDICTION

AIM:

To design and train a Recurrent Neural Network (RNN) using a Simple RNN layer to predict the next value in a time-series sequence generated from a sine wave.

ALGORITHM:

STEP 1: Import required libraries: tensorflow, matplotlib, Simple RNN and Dense.

STEP 2: Generate synthetic time-series data (sine wave) and create input-output pairs where each input sequence predicts its next value.

STEP 3: Reshape the data into 3D format [samples, timesteps, features] required by RNN layers.

STEP 4: Build the RNN model using a SimpleRNN layer with 50 units and tanh activation, followed by a Dense layer with 1 neuron for output.

STEP 5: Compile the model with the Adam optimizer and mean squared error (MSE) loss function.

STEP 6: Train the model for 20 epochs with a batch size of 32 and a validation split of 20%.

STEP 7: Evaluate the model by making predictions on sample inputs and comparing them with true values.

STEP 8: Plot the training and validation loss to visualize learning performance.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import Sequential
```

```

from tensorflow.keras.layers import SimpleRNN, Dense

def generate_sine_wave(seq_length, num_samples):
    X, y = [], []
    for _ in range(num_samples):
        start = np.random.rand() * 2 * np.pi
        xs = np.linspace(start, start + 3 * np.pi, seq_length + 1)
        data = np.sin(xs)
        X.append(data[:-1])
        y.append(data[-1])
    return np.array(X), np.array(y)

seq_length = 50
num_samples = 1000
X, y = generate_sine_wave(seq_length, num_samples)
X = X.reshape((num_samples, seq_length, 1))
model = Sequential([
    SimpleRNN(50, activation='tanh', input_shape=(seq_length, 1)),
    Dense(1) # Predict next value in sequence])
model.compile(optimizer='adam', loss='mse')
history = model.fit(X, y, epochs=20, batch_size=32, validation_split=0.2)
pred = model.predict(X[:10])
print("\nSample Predictions:")
for i in range(5):
    print(f"True: {y[i]:.3f}, Predicted: {pred[i][0]:.3f}")
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend()
plt.title("RNN Training Performance")
plt.show()

```

OUTPUT:

Epoch 1/20

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

25/25 ————— 1s 17ms/step - loss: 0.3049 - val_loss: 0.0153

Epoch 2/20

25/25 ————— 0s 8ms/step - loss: 0.0076 - val_loss: 6.1550e-04

Epoch 3/20

25/25 ————— 0s 10ms/step - loss: 6.1812e-04 - val_loss: 4.2186e-05

Epoch 4/20

25/25 ————— 0s 8ms/step - loss: 7.8323e-05 - val_loss: 2.5385e-05

Epoch 5/20

25/25 ————— 0s 8ms/step - loss: 2.4766e-05 - val_loss: 1.4559e-05

Epoch 6/20

25/25 ————— 0s 8ms/step - loss: 1.3221e-05 - val_loss: 1.2748e-05

Epoch 7/20

25/25 ————— 0s 10ms/step - loss: 1.2374e-05 - val_loss: 1.1299e-05

Epoch 8/20

25/25 ————— 0s 8ms/step - loss: 9.4861e-06 - val_loss: 9.9077e-06

Epoch 9/20

25/25 ————— 0s 8ms/step - loss: 8.6906e-06 - val_loss: 7.1489e-06

Epoch 10/20

25/25 ————— 0s 9ms/step - loss: 6.9545e-06 -
val_loss: 5.6677e-06

Epoch 11/20

25/25 ————— 0s 8ms/step - loss: 5.2897e-06 -
val_loss: 5.1961e-06

Epoch 12/20

25/25 ————— 0s 8ms/step - loss: 4.9279e-06 -
val_loss: 4.0503e-06

Epoch 13/20

25/25 ————— 0s 8ms/step - loss: 4.0215e-06 -
val_loss: 3.7501e-06

Epoch 14/20

25/25 ————— 0s 8ms/step - loss: 3.2531e-06 -
val_loss: 3.1022e-06

Epoch 15/20

25/25 ————— 0s 8ms/step - loss: 2.8326e-06 -
val_loss: 2.0488e-06

Epoch 16/20

25/25 ————— 0s 8ms/step - loss: 1.8347e-06 -
val_loss: 1.5976e-06

Epoch 17/20

25/25 ————— 0s 8ms/step - loss: 1.6108e-06 -
val_loss: 1.5111e-06

Epoch 18/20

25/25 ————— 0s 12ms/step - loss: 1.1025e-06 -
val_loss: 1.0988e-06

Epoch 19/20

25/25 ————— 0s 14ms/step - loss: 9.9462e-07 -
val_loss: 8.3697e-07

Epoch 20/20

25/25 ————— 0s 14ms/step - loss: 7.3977e-07 -
val_loss: 7.0010e-07

Sample Predictions:

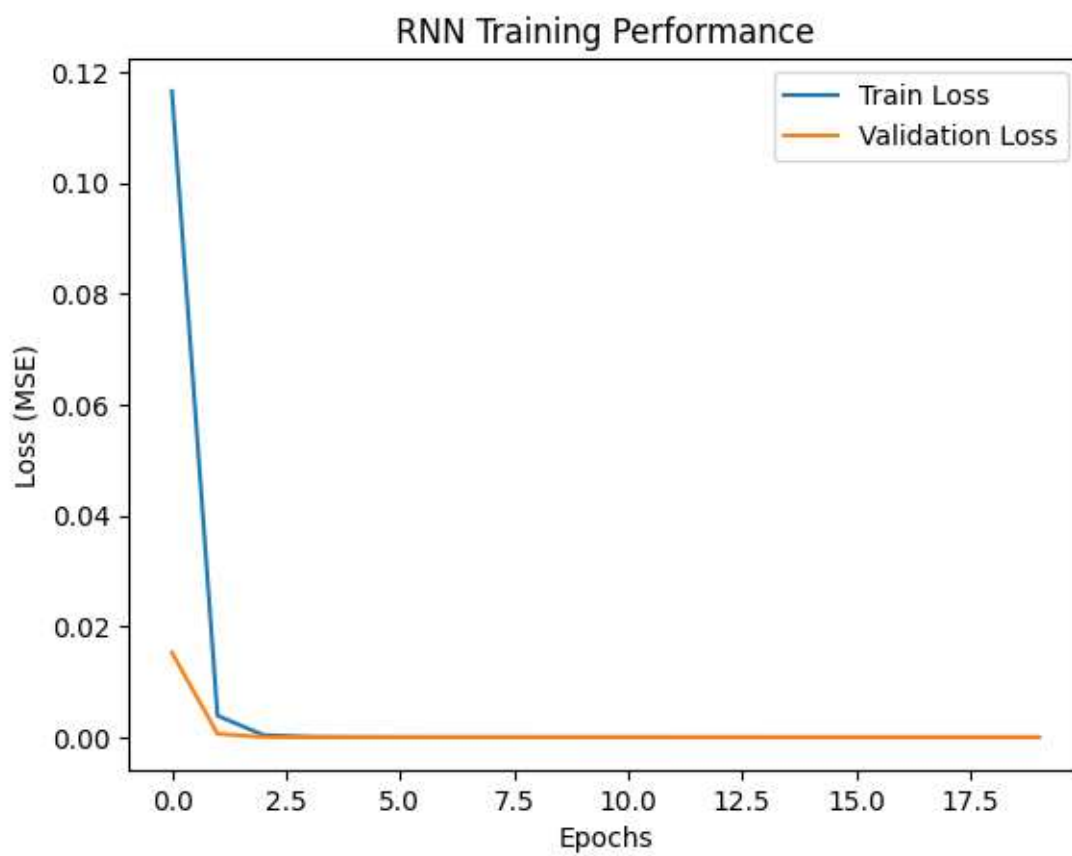
True: 0.391, Predicted: 0.391

True: 0.528, Predicted: 0.529

True: 0.432, Predicted: 0.433

True: -0.365, Predicted: -0.365

True: 0.599, Predicted: 0.600



COE (20)	
RECORD (20)	
VIVA (10)	
TOTAL (50)	

RESULT:

The RNN was successfully implemented and trained on synthetic sine-wave data. It accurately predicted the next time-step values, and the training plot showed decreasing loss, demonstrating the effectiveness of the RNN architecture for time-series forecasting tasks.

EX. NO: 07

DATE :

TEXT ANALYSIS USING NATURAL LANGUAGE PROCESSING

AIM:

To perform end-to-end text analysis cleaning, feature extraction, and visualization on a sample text dataset using Natural Language Processing techniques.

ALGORITHM:

STEP 1: Install required libraries (nltk, scikit-learn, pandas, matplotlib, and wordcloud). Import pandas for data handling, re for regex, nltk for NLP utilities, TfidfVectorizer from sklearn for feature extraction, matplotlib.pyplot for plotting, and WordCloud for visualization.

STEP 2: Create or load a text dataset into a Pandas DataFrame for analysis.

STEP 3: Text preprocessing converts text to lowercase, removes punctuation and special characters, tokenizes into words, filters out stopwords, and rejoins the cleaned tokens into a processed string for analysis.

STEP 4: Combine all cleaned text and compute word frequencies using pandas.Series.value_counts() to identify the most frequent words.

STEP 4: Use TfidfVectorizer to transform the cleaned text into numerical features, capturing the importance of words in each document relative to the entire dataset. Extract and display top TF-IDF words per document.

STEP 5: Create a WordCloud from the cleaned text to visually highlight frequently occurring words in the dataset.

PROGRAM:

```
!pip install nltk scikit-learn pandas matplotlib wordcloud --quiet
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
import matplotlib.pyplot as plt
from wordcloud import WordCloud
nltk.download('stopwords')
data = {
    'text': [
        "The movie had stunning visuals but the plot was weak.",
        "Customer service was prompt and very helpful.",
        "I found the book to be quite boring and slow-paced.",
        "Excellent craftsmanship and attention to detail in this product.",
        "The app crashes every time I try to open it.",
        "Had an amazing dinner at the new Italian restaurant downtown.",
        "Terrible experience. I will not return.",
        "The software update improved performance significantly.",
        "Delivery was late and the package was damaged.",
        "Great user interface and very intuitive controls.",
        "Music quality is outstanding, especially the bass.",
        "Not impressed. Expected more for the price.",
        "Enjoyed the hiking trail — beautiful scenery and fresh air.",
        "Keyboard keys are too stiff and unresponsive.",
        "Friendly staff and a clean environment at the clinic.",
        "The laptop heats up quickly when gaming.",
        "Loved the plot twists in the final episodes!",
        "Battery life is shorter than advertised."
```

```

    ]]
df = pd.DataFrame(data)
print("Original Data:")
print(df)
stop_words = set(stopwords.words('english'))
def preprocess(text):
    text = text.lower()          # lowercase
    text = re.sub(r'^a-z\s', '', text) # remove punctuation
    tokens = text.split()        # simple tokenization
    tokens = [word for word in tokens if word not in stop_words] # remove stopwords
    return " ".join(tokens)
df['clean_text'] = df['text'].apply(preprocess)
print("\nCleaned Text:")
print(df[['text', 'clean_text']])
all_words = ' '.join(df['clean_text']).split()
word_freq = pd.Series(all_words).value_counts()
print("\nTop 10 Most Frequent Words:")
print(word_freq.head(10))
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['clean_text'])
feature_names = vectorizer.get_feature_names_out()
print("\nTop 5 TF-IDF Words per Document:")
for i, doc in enumerate(df['clean_text']):
    tfidf_scores = X[i].toarray()[0]
    top_indices = tfidf_scores.argsort()[-5:][::-1]
    top_words = [(feature_names[idx], tfidf_scores[idx]) for idx in top_indices if
tfidf_scores[idx] > 0]
    print(f"Document {i+1}: {top_words}")
text_combined = ' '.join(df['clean_text'])
wordcloud = WordCloud(width=800, height=400,
background_color='white').generate(text_combined)

```

```
plt.figure(figsize=(12,6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("WordCloud of All Documents", fontsize=16)
plt.show()
```

OUTPUT:

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Unzipping corpora/stopwords.zip.

Original Data:

text

- 0 The movie had stunning visuals but the plot wa...
- 1 Customer service was prompt and very helpful.
- 2 I found the book to be quite boring and slow-p...
- 3 Excellent craftsmanship and attention to detai...
- 4 The app crashes every time I try to open it.
- 5 Had an amazing dinner at the new Italian resta...
- 6 Terrible experience. I will not return.
- 7 The software update improved performance signi...
- 8 Delivery was late and the package was damaged.
- 9 Great user interface and very intuitive controls.
- 10 Music quality is outstanding, especially the b...
- 11 Not impressed. Expected more for the price.
- 12 Enjoyed the hiking trail — beautiful scenery a...
- 13 Keyboard keys are too stiff and unresponsive.
- 14 Friendly staff and a clean environment at the ...
- 15 The laptop heats up quickly when gaming.
- 16 Loved the plot twists in the final episodes!
- 17 Battery life is shorter than advertised.

Cleaned Text:

text \

- 0 The movie had stunning visuals but the plot wa...
- 1 Customer service was prompt and very helpful.
- 2 I found the book to be quite boring and slow-p...
- 3 Excellent craftsmanship and attention to detai...
- 4 The app crashes every time I try to open it.
- 5 Had an amazing dinner at the new Italian resta...
- 6 Terrible experience. I will not return.
- 7 The software update improved performance signi...
- 8 Delivery was late and the package was damaged.
- 9 Great user interface and very intuitive controls.
- 10 Music quality is outstanding, especially the b...
- 11 Not impressed. Expected more for the price.
- 12 Enjoyed the hiking trail — beautiful scenery a...
- 13 Keyboard keys are too stiff and unresponsive.
- 14 Friendly staff and a clean environment at the ...
- 15 The laptop heats up quickly when gaming.
- 16 Loved the plot twists in the final episodes!
- 17 Battery life is shorter than advertised.

clean_text

- 0 movie stunning visuals plot weak
- 1 customer service prompt helpful
- 2 found book quite boring slowpaced
- 3 excellent craftsmanship attention detail product
- 4 app crashes every time try open
- 5 amazing dinner new italian restaurant downtown
- 6 terrible experience return

7 software update improved performance significa...

8 delivery late package damaged

9 great user interface intuitive controls

10 music quality outstanding especially bass

11 impressed expected price

12 enjoyed hiking trail beautiful scenery fresh air

13 keyboard keys stiff unresponsive

14 friendly staff clean environment clinic

15 laptop heats quickly gaming

16 loved plot twists final episodes

17 battery life shorter advertised

Top 10 Most Frequent Words:

plot 2

movie 1

stunning 1

visuals 1

weak 1

customer 1

service 1

prompt 1

helpful 1

found 1

Name: count, dtype: int64

Top 5 TF-IDF Words per Document:

Document 1: [('weak', np.float64(0.4580541841950169)), ('visuals', np.float64(0.4580541841950169)), ('stunning', np.float64(0.4580541841950169)), ('movie', np.float64(0.4580541841950169)), ('plot', np.float64(0.400930738863647))]

Document 2: [('service', np.float64(0.5)), ('customer', np.float64(0.5)), ('helpful', np.float64(0.5)), ('prompt', np.float64(0.5))]

Document 3: [('slowpaced', np.float64(0.4472135954999579)), ('boring', np.float64(0.4472135954999579)), ('book', np.float64(0.4472135954999579)), ('quite', np.float64(0.4472135954999579)), ('found', np.float64(0.4472135954999579))]

Document 4: [('excellent', np.float64(0.4472135954999579)), ('detail', np.float64(0.4472135954999579)), ('attention', np.float64(0.4472135954999579)), ('product', np.float64(0.4472135954999579)), ('craftsmanship', np.float64(0.4472135954999579))]

Document 5: [('time', np.float64(0.408248290463863)), ('try', np.float64(0.408248290463863)), ('app', np.float64(0.408248290463863)), ('open', np.float64(0.408248290463863)), ('every', np.float64(0.408248290463863))]

Document 6: [('dinner', np.float64(0.408248290463863)), ('italian', np.float64(0.408248290463863)), ('new', np.float64(0.408248290463863)), ('restaurant', np.float64(0.408248290463863)), ('amazing', np.float64(0.408248290463863))]

Document 7: [('return', np.float64(0.5773502691896258)), ('terrible', np.float64(0.5773502691896258)), ('experience', np.float64(0.5773502691896258))]

Document 8: [('update', np.float64(0.4472135954999579)), ('significantly', np.float64(0.4472135954999579)), ('software', np.float64(0.4472135954999579)), ('performance', np.float64(0.4472135954999579)), ('improved', np.float64(0.4472135954999579))]

Document 9: [('damaged', np.float64(0.5)), ('delivery', np.float64(0.5)), ('late', np.float64(0.5)), ('package', np.float64(0.5))]

Document 10: [('user', np.float64(0.4472135954999579)), ('intuitive', np.float64(0.4472135954999579)), ('great', np.float64(0.4472135954999579)), ('interface', np.float64(0.4472135954999579)), ('controls', np.float64(0.4472135954999579))]

Document 11: [('especially', np.float64(0.4472135954999579)), ('music', np.float64(0.4472135954999579)), ('outstanding', np.float64(0.4472135954999579)), ('quality', np.float64(0.4472135954999579)), ('bass', np.float64(0.4472135954999579))]

Document 12: [('impressed', np.float64(0.5773502691896258)), ('expected', np.float64(0.5773502691896258)), ('price', np.float64(0.5773502691896258))]

Document 13: [('scenery', np.float64(0.3779644730092272)), ('trail', np.float64(0.3779644730092272)), ('beautiful', np.float64(0.3779644730092272)), ('hiking', np.float64(0.3779644730092272)), ('fresh', np.float64(0.3779644730092272))]

Document 14: [('stiff', np.float64(0.5)), ('unresponsive', np.float64(0.5)), ('keys', np.float64(0.5)), ('keyboard', np.float64(0.5))]

Document 15: [('staff', np.float64(0.4472135954999579)), ('clean', np.float64(0.4472135954999579)), ('clinic', np.float64(0.4472135954999579)), ('friendly', np.float64(0.4472135954999579)), ('environment', np.float64(0.4472135954999579))]

Document 16: [('laptop', np.float64(0.5)), ('gaming', np.float64(0.5)), ('heats', np.float64(0.5)), ('quickly', np.float64(0.5))]

Document 17: [('twists', np.float64(0.4580541841950169)), ('episodes', np.float64(0.4580541841950169)), ('final', np.float64(0.4580541841950169)), ('loved', np.float64(0.4580541841950169)), ('plot', np.float64(0.400930738863647))]

Document 18: [('shorter', np.float64(0.5)), ('advertised', np.float64(0.5)), ('battery', np.float64(0.5)), ('life', np.float64(0.5))]



COE (20)	
RECORD (20)	
VIVA (10)	
TOTAL (50)	

RESULT:

The NLP pipeline successfully cleaned and normalized the raw text, identified the most frequent terms along with top TF-IDF features for each document, and generated a WordCloud that visually highlights the most significant words across the dataset.

Exp No: 8 (a)

Date:

DEEPDREAM IMAGE GENERATION

AIM: -

To implement the deepdream image generation.

ALGORITHM: -

Step 1: Install and Import Libraries

- Install required Python libraries: tensorflow, matplotlib, Pillow.
- Import libraries for:
 - TensorFlow (for deep learning models),
 - NumPy (for numerical operations),
 - PIL (for image handling),
 - Matplotlib (for displaying images),
 - Google Colab files (for uploading images).

Step 2: Load Pre-trained CNN

- Load a **pre-trained InceptionV3 model** without the top classification layers (include_top=False).
- Select the **layers to enhance** for DeepDream. Early layers capture textures; deeper layers capture objects/faces.
- Create a **feature extraction model** using the selected layers.

Step 3: Define Utility Functions

- load_img(path, max_dim): Loads an image, resizes it to a maximum dimension while maintaining aspect ratio, and adds a batch dimension [1, H, W, 3].
- deprocess(img): Converts processed tensor values ([-1,1]) back to standard image pixel values [0,255].
- show_img(img): Displays an image using Matplotlib.

Step 4: Define DeepDream Functions

- `calc_loss(img, model)`: Passes the image through the selected CNN layers and calculates the **loss** as the sum of mean activations from all layers.
- `deepdream_step(img, model, step_size)`:
 - Computes gradients of the loss with respect to the image.
 - Normalizes gradients and updates the image using **gradient ascent**:
$$\text{img} = \text{img} + \text{grads} * \text{step_size}$$
 - Clips image values to $[-1,1]$ to maintain valid pixel range.
- `run_deep_dream(img, steps, step_size)`:
 - Iteratively applies `deepdream_step` for a fixed number of steps.
 - Prints intermediate loss every 20 steps.

Step 5: Upload and Preprocess Image

- Upload an image via Colab.
- Load and preprocess the image using `load_img()`.
- Display the original image.

Step 6: Run DeepDream

- Convert the image to the expected range $[-1,1]$.
- Pass the image through `run_deep_dream` to iteratively enhance patterns.

Step 7: Postprocess and Save

- Convert the output tensor back to pixel values using `deprocess()`.
- Remove the extra batch dimension using `np.squeeze()`.
- Display the final DeepDream image.
- Save the image as "deepdream_result.jpg".

Step 8: Experimentation Options

- Adjust **steps** (more → stronger effect).
- Adjust **step size** (larger → more psychedelic patterns).
- Change **layers** to enhance different features (textures vs objects).

PROGRAM: -

```
#Import libraries
import tensorflow as tf
import numpy as np
import PIL.Image
import matplotlib.pyplot as plt
from google.colab import files

#Load a pre-trained InceptionV3 model
base_model = tf.keras.applications.InceptionV3(include_top=False, weights='imagenet')

# Choose layers to enhance (experiment with these)
names = ['mixed3', 'mixed5']
layers = [base_model.get_layer(name).output for name in names]

# Build the feature extraction model
dream_model = tf.keras.Model(inputs=base_model.input, outputs=layers)

# Utility functions
def deprocess(img):
    img = 255 * (img + 1.0) / 2.0
    return np.uint8(img)

def load_img(path, max_dim=512):
    img = tf.io.read_file(path)
    img = tf.image.decode_image(img, channels=3)
    img = tf.image.convert_image_dtype(img, tf.float32)

    # Resize while keeping aspect ratio
    shape = tf.cast(tf.shape(img)[:2], tf.float32)
    scale = max_dim / max(shape)
    new_shape = tf.cast(shape * scale, tf.int32)
```

```

img = tf.image.resize(img, new_shape)

# Add batch dimension
img = tf.expand_dims(img, axis=0)
return img

def show_img(img):
    plt.figure(figsize=(8,8))
    plt.imshow(np.squeeze(img))
    plt.axis('off')
    plt.show()

# DeepDream functions
def calc_loss(img, model):
    layer_activations = model(img)
    if len(layer_activations) == 1:
        layer_activations = [layer_activations]
    losses = [tf.reduce_mean(act) for act in layer_activations]
    return tf.reduce_sum(losses)

@tf.function
def deepdream_step(img, model, step_size):
    with tf.GradientTape() as tape:
        tape.watch(img)
        loss = calc_loss(img, model)
    grads = tape.gradient(loss, img)
    grads /= tf.math.reduce_std(grads) + 1e-8
    img = img + grads * step_size
    img = tf.clip_by_value(img, -1, 1)
    return img, loss

def run_deep_dream(img, steps=100, step_size=0.01):
    for step in range(steps):
        img, loss = deepdream_step(img, dream_model, step_size)

```

```
        if step % 20 == 0:
            print(f"Step {step}, Loss {loss}")
    return img

#Upload image in Colab
uploaded = files.upload()
image_path = list(uploaded.keys())[0]

# Load and preprocess image
original_img = load_img(image_path)
show_img(original_img)

# Run DeepDream
dream_img = run_deep_dream(original_img*2-1, steps=100, step_size=0.01)

# Show and save result
dream_img = deprocess(dream_img.numpy())

# Remove batch dimension
dream_img = np.squeeze(dream_img, axis=0)

# Show and save
show_img(dream_img)
result = PIL.Image.fromarray(dream_img)
result.save("deepdream_result.jpg")
print("DeepDream image saved as deepdream_result.jpg")
```

OUTPUT: -

Choose files output_step_0.png
output_step_0.png(image/png) - 348356 bytes, last modified: 29/09/2025 - 100% done
Saving output_step_0.png to output_step_0 (2).png



Step 0, Loss 0.5668067336082458
Step 20, Loss 1.4887861013412476
Step 40, Loss 1.8054102659225464
Step 60, Loss 1.9969892501831055
Step 80, Loss 2.1354920864105225



RESULT: -

Thus, the implementation of deepdream image generation is successfully executed.

Exp No: 08 (b)

Date:

Neural Style Transfer

AIM: -

To implement the neural style transfer

ALGORITHM: -

1. Inputs

- **Content Image (C)** → carries the structure, objects, and layout you want to preserve.
- **Style Image (S)** → carries textures, colors, and artistic style to apply.

2. Image Transformation Network

- This is a **trainable feed-forward CNN** (not VGG).
- It takes the **Content Image C** as input and produces an **Output Image Y** (stylized).
- Instead of optimizing pixels directly (like Gatys' original NST), here we train this network to generate stylized images quickly.

3. VGG Network (Feature Extractor)

- A **fixed, pre-trained VGG network** (e.g., VGG19 trained on ImageNet).
- It is **not trained** here — only used to extract features.
- It processes:
 - **Content Image (C)** → produces **content representation**.
 - **Style Image (S)** → produces **style representation** (via Gram matrices).
 - **Output Image (Y)** → produces both **content & style representations**.

4. Loss Calculation

- Compare the **Output Image's** features with both **Content** and **Style** features:
 - **Content Loss** = difference between content features of C and Y.
 - **Style Loss** = difference between style features (Gram matrices) of S and Y.
- Both losses are combined:

$$L_{total} = \alpha \cdot L_{content} + \beta \cdot L_{style}$$

$$L_{total} = \alpha \cdot L_{content} + \beta \cdot L_{style}$$

5. Optimization

- The **Image Transformation Network** is trained to minimize this loss.
- Over time, the network learns to generate stylized output images **in one forward pass** (fast style transfer).

PROGRAM: -

```
import torch

import torch.nn as nn

import torch.optim as optim

import torchvision.transforms as transforms

import torchvision.models as models

from PIL import Image

import matplotlib.pyplot as plt

import copy

import os

----- User settings -----

content_img_path = "/content/drive/MyDrive/Colab Notebooks/Cat03.jpg"

style_img_path = "/content/drive/MyDrive/Colab Notebooks/wave.jpg"

output_dir = "nst_outputs"

imgsize = 512          # set 512 for GPU, 256 for CPU if memory limited

num_steps = 300        # total optimization steps

style_weight = 1e6      # style loss weight

content_weight = 1e0     # content loss weight
```

```

use_gpu = torch.cuda.is_available()

device = torch.device("cuda" if use_gpu else "cpu")

# -----

if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# image loader / preprocessor
loader = transforms.Compose([
    transforms.Resize(imsize),
    transforms.CenterCrop(imsize),
    transforms.ToTensor()
])

unloader = transforms.ToPILImage()

def load_image(path):
    image = Image.open(path).convert('RGB')
    image = loader(image).unsqueeze(0) # add batch dimension
    return image.to(device, torch.float)

def imsave(tensor, filename):
    image = tensor.cpu().clone().detach()
    image = image.squeeze(0)
    image = unloader(image)
    image.save(filename)

# Gram matrix for style representation

def gram_matrix(input_tensor):

```

```

b, ch, h, w = input_tensor.size()

features = input_tensor.view(b * ch, h * w)

G = torch.mm(features, features.t())

return G.div(b * ch * h * w)

# Build a VGG model that returns activations at a set of layers

cnn = models.vgg19(pretrained=True).features.to(device).eval()

Freeze params (we won't train CNN)

for param in cnn.parameters():
    param.requires_grad = False

# Content and style layer choices (typical)

content_layers = ['21'] # conv4_2

style_layers = ['0', '5', '10', '19', '28'] # conv1_1, conv2_1, conv3_1, conv4_1, conv5_1

# Helper to extract features

def get_features(x, model, layers):

    features = {}

    i = 0

    for layer in model.children():

        x = layer(x)

        if str(i) in layers:

            features[str(i)] = x

        i += 1

    return features

# Load images

content_img = load_image(content_img_path)

```

```

style_img = load_image(style_img_path)

assert content_img.size() == style_img.size(), "Style and content images must be same size"

# Initialize target image as a copy of content or random noise
target = content_img.clone().requires_grad_(True).to(device)

# Compute features for content and style
content_feats = get_features(content_img, cnn, content_layers)
style_feats = get_features(style_img, cnn, style_layers)
style_grams = {layer: gram_matrix(style_feats[layer]) for layer in style_feats}

# Loss: content + style
mse_loss = nn.MSELoss()

# Use L-BFGS or Adam
optimizer = optim.LBFGS([target]) # good for this problem
# optimizer = optim.Adam([target], lr=0.02) # alternative

print("Starting optimization on", device)

run = [0]

while run[0] <= num_steps:

    def closure():
        # clamp target to valid pixel range
        with torch.no_grad():
            target.clamp_(0, 1)

```

```
optimizer.zero_grad()

target_feats = get_features(target, cnn, content_layers + style_layers)

# content loss

content_loss = 0.0

for layer in content_layers:

    target_f = target_feats[layer]

    content_f = content_feats[layer]

    content_loss += mse_loss(target_f, content_f)

# style loss

style_loss = 0.0

for layer in style_layers:

    target_f = target_feats[layer]

    target_gram = gram_matrix(target_f)

    style_gram = style_grams[layer]

    style_loss += mse_loss(target_gram, style_gram)

total_loss = content_weight * content_loss + style_weight * style_loss

total_loss.backward()

# print & save intermediate

if run[0] % 50 == 0:
```

```

print("Step {}: Total Loss: {:.4e}, Content: {:.4e}, Style: {:.4e}".format(
    run[0], total_loss.item(), content_loss.item(), style_loss.item()))

out_path = os.path.join(output_dir, f"output_step_{run[0]}.png")

# clamp before saving

with torch.no_grad():

    tmp = target.clone().detach()

    tmp.clamp_(0,1)

    imsave(tmp, out_path)


run[0] += 1

return total_loss

optimizer.step(closure)

# final clamp and save

with torch.no_grad():

    target.clamp_(0,1)

    imsave(target, os.path.join(output_dir, "final_output.png"))

print("Finished. Final image saved to", os.path.join(output_dir, "final_output.png"))

```

OUTPUT: -



COE (20)	
RECORD (20)	
VIVA (10)	
TOTAL (50)	

RESULT: -

Thus, the implementation of neural style transfer is successfully executed.

Exp No: 09

Date:

GENERATING SYNTHETIC IMAGES USING VAE

AIM: -

To implement the VAE by generating the synthetic images

ALGORITHM: -

1. Initialize Parameters

- Set latent dimension size, batch size, learning rate, and number of epochs.
- Choose device (GPU if available, else CPU).
- Create an output directory to save generated images.

2. Prepare Dataset

- Load MNIST dataset (28×28 grayscale images).
- Apply transformation (ToTensor) to convert images into tensors scaled between [0,1].
- Use DataLoader to batch and shuffle the dataset.

3. Define VAE Model

- **Encoder:**
 - Flatten input image.
 - Pass through fully connected layers with activation (ReLU).
 - Output two vectors: mean (μ) and log-variance ($\log\sigma^2$).
- **Reparameterization Trick:**
 - Sample $\epsilon \sim N(0, I)$.
 - Compute latent vector $z = \mu + \sigma \cdot \epsilon$.
- **Decoder:**
 - Pass latent vector z through fully connected layers with activation (ReLU).
 - Output a reconstructed image using Sigmoid activation (ensures values between 0–1).

4. Define Loss Function

- **Reconstruction Loss (BCE):** Measures pixel-wise difference between input and reconstructed image.
- **KL Divergence:** Forces latent distribution $q(z|x)$ to be close to the standard normal distribution $p(z)$.
- Total Loss = BCE + KL Divergence.

5. Training Loop

- For each epoch:
 1. For each batch:
 - Forward pass: input image $\rightarrow$ encoder $\rightarrow$ latent $z \rightarrow$ decoder $\rightarrow$ reconstruction.
 - Compute loss (BCE + KL).
 - Backpropagate and update model parameters using Adam optimizer.
 2. Print average loss after each epoch.

6. Image Generation

- After training:
 - Sample random latent vectors $z \sim N(0, I)$.
 - Pass z through decoder to generate synthetic images.
 - Save images to output folder.

7. Reconstruction

- Pass real images through encoder $\rightarrow$ reparameterization $\rightarrow$ decoder.
- Compare original images with reconstructions and save output.

8. Interpolation in Latent Space

- Choose two real images.
- Encode them into latent vectors z_1 and z_2 .
- Linearly interpolate between z_1 and z_2 .
- Decode intermediate z values to generate a smooth transition of images.

PROGRAM: -

```
pip install torch torchvision tqdm matplotlib
```

```
# vae_mnist.py
```

```
import os
```

```
import math
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torchvision import datasets, transforms, utils
```

```
from torch.utils.data import DataLoader
```

```
from tqdm import tqdm
```

```
# -----
```

```
# Config / Hyperparams
```

```
# -----
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
batch_size = 128
```

```
epochs = 20
```

```
latent_dim = 20
```

```
lr = 1e-3
```

```
log_interval = 200
```

```
output_dir = "vae_outputs"
```

```
os.makedirs(output_dir, exist_ok=True)
```

```

# -----

# Dataset

# -----

transform = transforms.Compose([

    transforms.ToTensor()

])

train_dataset = datasets.MNIST(root="data", train=True, download=True,
transform=transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
num_workers=2, pin_memory=True)

# -----

# VAE Model (simple conv-free for MNIST)

# -----

class VAE(nn.Module):

    def __init__(self, latent_dim=20):

        super().__init__()

        # encoder: simple MLP

        self.enc = nn.Sequential(

            nn.Flatten(),

            nn.Linear(28*28, 400),

            nn.ReLU(),

        )

        self.fc_mu = nn.Linear(400, latent_dim)

```

```
self.fc_logvar = nn.Linear(400, latent_dim)
```

```
# decoder
```

```
self.dec_fc = nn.Sequential(  
    nn.Linear(latent_dim, 400),  
    nn.ReLU(),  
    nn.Linear(400, 28*28),  
    nn.Sigmoid() # outputs in [0,1]  
)
```

```
def encode(self, x):
```

```
    h = self.enc(x)  
    mu = self.fc_mu(h)  
    logvar = self.fc_logvar(h)  
    return mu, logvar
```

```
def reparameterize(self, mu, logvar):
```

```
    # reparameterization trick  
    std = torch.exp(0.5 * logvar)  
    eps = torch.randn_like(std)  
    return mu + eps * std
```

```
def decode(self, z):
```

```
    x_recon = self.dec_fc(z)
```

```

        x_recon = x_recon.view(-1, 1, 28, 28)

    return x_recon

def forward(self, x):

    mu, logvar = self.encode(x)

    z = self.reparameterize(mu, logvar)

    x_recon = self.decode(z)

    return x_recon, mu, logvar

# -----

# Loss: BCE (reconstruction) + KL

# -----

def vae_loss(recon_x, x, mu, logvar):

    # reconstruction term (binary cross entropy per pixel)

    BCE = nn.functional.binary_cross_entropy(recon_x, x, reduction='sum')

    # KL divergence between posterior  $N(\mu, \sigma^2)$  and prior  $N(0,1)$ 

    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())

    return BCE + KLD, BCE, KLD

# -----

# Training

# -----

model = VAE(latent_dim=latent_dim).to(device)

optimizer = optim.Adam(model.parameters(), lr=lr)

```

```

def train(epoch):

    model.train()

    train_loss = 0.0

    pbar = tqdm(enumerate(train_loader), total=len(train_loader))

    for batch_idx, (data, _) in pbar:

        data = data.to(device)

        optimizer.zero_grad()

        recon_batch, mu, logvar = model(data)

        loss, bce, kld = vae_loss(recon_batch, data, mu, logvar)

        loss.backward()

        train_loss += loss.item()

        optimizer.step()

        if batch_idx % log_interval == 0:

            pbar.set_description(f'Epoch {epoch} Batch {batch_idx} Loss
{loss.item()/len(data):.4f} BCE {bce/len(data):.4f} KLD {kld/len(data):.4f}')

    avg_loss = train_loss / len(train_loader.dataset)

    print(f'====> Epoch {epoch} Average loss: {avg_loss:.4f}')

# -----

# Utilities: sample and interpolate

# -----

@torch.no_grad()

def sample_and_save(epoch, n=64):

```

```

model.eval()

z = torch.randn(n, latent_dim).to(device)

samples = model.decode(z).cpu()

utils.save_image(samples, f'{output_dir}/samples_epoch_{epoch}.png', nrow=8,
padding=2)

```

```

@torch.no_grad()

def reconstruct_and_save(x, filename="recon.png", n=8):

    model.eval()

    x = x.to(device)[:n]

    recon, _, _ = model(x)

    # concat originals and reconstructions

    comparison = torch.cat([x.cpu(), recon.cpu()])

    utils.save_image(comparison, filename, nrow=n, padding=2)

```

```

@torch.no_grad()

def interpolate_and_save(index_a, index_b, dataset, steps=8, filename="interpolate.png"):

    # pick two images from dataset and encode them, then linearly interpolate in z

    model.eval()

    xa, _ = dataset[index_a]

    xb, _ = dataset[index_b]

    xa = xa.unsqueeze(0).to(device)

    xb = xb.unsqueeze(0).to(device)

    mu_a, logvar_a = model.encode(xa)

    mu_b, logvar_b = model.encode(xb)

```

```

za = mu_a # use mean for deterministic interpolation

zb = mu_b

all_imgs = []

for alpha in torch.linspace(0, 1, steps):

    z = (1-alpha) * za + alpha * zb

    img = model.decode(z).cpu()

    all_imgs.append(img)

grid = torch.cat(all_imgs, dim=0)

utils.save_image(grid, filename, nrow=steps, padding=2)


# -----

# Main training loop

# -----

if __name__ == "__main__":

    for epoch in range(1, epochs+1):

        train(epoch)

        sample_and_save(epoch, n=64)

    # save model

    torch.save(model.state_dict(), os.path.join(output_dir, "vae_mnist.pth"))

    print("Training complete. Samples saved in", output_dir)

    # example: save reconstruction comparison (use first batch)

    test_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)

    data_iter = iter(test_loader)

    x, _ = next(data_iter)

```

```

reconstruct_and_save(x, filename=os.path.join(output_dir, "reconstruction.png"), n=8)

# example: interpolation between two dataset items (indices)

interpolate_and_save(1, 10, train_dataset, steps=10, filename=os.path.join(output_dir,
"interp.png"))

print("Saved reconstruction and interpolation images.")

```

OUTPUT: -

```

Epoch 7 Batch 400 Loss 107.5740 BCE 82.8399 KLD 24.7341: 100%|██████████| 469/469 [00:07<00:00, 62.22it/s]====> Epoch 7 Average loss: 108.0125
Epoch 8 Batch 400 Loss 107.7619 BCE 82.1332 KLD 25.6287: 100%|██████████| 469/469 [00:07<00:00, 59.92it/s]====> Epoch 8 Average loss: 107.3202
Epoch 9 Batch 400 Loss 108.5338 BCE 83.0104 KLD 25.5233: 100%|██████████| 469/469 [00:06<00:00, 68.04it/s]====> Epoch 9 Average loss: 106.7956
Epoch 10 Batch 400 Loss 104.7559 BCE 79.8464 KLD 24.9095: 100%|██████████| 469/469 [00:07<00:00, 60.23it/s]====> Epoch 10 Average loss: 106.3813
Epoch 11 Batch 400 Loss 106.6543 BCE 81.5160 KLD 25.1383: 100%|██████████| 469/469 [00:07<00:00, 64.21it/s]====> Epoch 11 Average loss: 106.0447
Epoch 12 Batch 400 Loss 104.2354 BCE 79.0394 KLD 25.1961: 100%|██████████| 469/469 [00:07<00:00, 64.10it/s]====> Epoch 12 Average loss: 105.6968
Epoch 13 Batch 400 Loss 101.7215 BCE 77.1955 KLD 24.5260: 100%|██████████| 469/469 [00:07<00:00, 61.10it/s]====> Epoch 13 Average loss: 105.4054
Epoch 14 Batch 400 Loss 106.5979 BCE 80.6492 KLD 25.9487: 100%|██████████| 469/469 [00:06<00:00, 68.58it/s]====> Epoch 14 Average loss: 105.1540
Epoch 15 Batch 400 Loss 103.5979 BCE 78.6691 KLD 24.9289: 100%|██████████| 469/469 [00:07<00:00, 60.62it/s]====> Epoch 15 Average loss: 104.9338
Epoch 16 Batch 400 Loss 105.4352 BCE 80.3764 KLD 25.0587: 100%|██████████| 469/469 [00:07<00:00, 66.71it/s]====> Epoch 16 Average loss: 104.7524
Epoch 17 Batch 400 Loss 102.2124 BCE 77.9314 KLD 24.2810: 100%|██████████| 469/469 [00:07<00:00, 62.50it/s]====> Epoch 17 Average loss: 104.5408
Epoch 18 Batch 400 Loss 103.1426 BCE 77.3923 KLD 25.7503: 100%|██████████| 469/469 [00:07<00:00, 60.84it/s]====> Epoch 18 Average loss: 104.3621
Epoch 19 Batch 400 Loss 102.5561 BCE 77.5128 KLD 25.0433: 100%|██████████| 469/469 [00:06<00:00, 67.60it/s]====> Epoch 19 Average loss: 104.1620
Epoch 20 Batch 400 Loss 103.9708 BCE 78.0564 KLD 25.9145: 100%|██████████| 469/469 [00:07<00:00, 61.34it/s]
====> Epoch 20 Average loss: 104.0347
Training complete. Samples saved in vae_outputs
Saved reconstruction and interpolation images.

```

COE (20)	
RECORD (20)	
VIVA (10)	
TOTAL (50)	

RESULT: -

Thus, the implementation of generating the synthetic image using VAE is executed successfully.

Exp No: 10

Date:

SYNTHETIC IMAGE GENERATION USING GAN

AIM: -

To implement the synthetic image generation using GAN

ALGORITHM: -

1. Initialize Parameters

- Set **batch size**, **image size (28×28 for MNIST)**, **latent vector size (nz)**, **learning rate (lr)**, and number of epochs.
- Choose device (GPU if available, else CPU).

2. Prepare Dataset

- Use **MNIST dataset** (grayscale handwritten digits).
- Apply **transforms**:
 - Convert images to tensors.
 - Normalize pixel values to range [-1, 1].
- Load dataset with a DataLoader.

3. Define Generator (G)

- Input: Random noise vector (latent space, size 100).
- Layers:
 1. **ConvTranspose2D** → Expand noise to feature maps.
 2. **BatchNorm + ReLU** → Normalize and activate.
 3. More **ConvTranspose2D** layers → Upsample to 28×28.
 4. Final layer: **ConvTranspose2D** → **Tanh()** → Output fake image in [-1,1].
- Output: Synthetic image (shape = 1×28×28).

4. Define Discriminator (D)

- Input: Real or fake image ($1 \times 28 \times 28$).
- Layers:
 1. **Conv2D** → Extract features from image.
 2. **LeakyReLU** → Non-linear activation.
 3. **Conv2D + BatchNorm + LeakyReLU** → Deep features.
 4. **Flatten + Linear + Sigmoid** → Output probability (real=1, fake=0).
- Output: Scalar (real/fake prediction).

5. Initialize Training Components

- Loss function: **Binary Cross-Entropy Loss (BCELoss)**.
- Optimizers: **Adam** for both G and D.
- Create a **fixed noise vector** for monitoring generator progress.

6. Training Loop (for each epoch)

(a) Train Discriminator (D):

1. Feed **real images** → predict real probability.
 - Compute **loss_real** against target label = 1.
 - Backpropagate.
2. Generate **fake images** using Generator.
 - Feed fake images into D.
 - Compute **loss_fake** against target label = 0.
 - Backpropagate.
3. Update D weights (gradient descent).

(b) Train Generator (G):

1. Generate fake images from random noise.
2. Pass them into D, but this time the target label = 1 (trick D).
3. Compute **loss_G** and backpropagate.
4. Update G weights.

7. Monitor Progress

- Print losses for D and G each epoch.
- Generate sample fake images using **fixed noise**.
- Display them to visually check improvements.

8. Output

- After training, Generator can take random noise and produce **synthetic handwritten digit images** that look realistic.

PROGRAM: -

```
pip install torch torchvision tqdm
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# ===== Config =====
batch_size = 128
image_size = 28
nz = 100 # size of latent vector
num_epochs = 5
lr = 0.0002
beta1 = 0.5
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# ===== Data =====
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # Normalize to [-1, 1]
])

dataset = torchvision.datasets.MNIST(root='./data', train=True, transform=transform,
download=True)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=batch_size, shuffle=True)

# ===== Generator =====
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.main = nn.Sequential(
            nn.ConvTranspose2d(nz, 128, 7, 1, 0, bias=False),
            nn.BatchNorm2d(128),
```

```

        nn.ReLU(True),

        nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False),
        nn.BatchNorm2d(64),
        nn.ReLU(True),

        nn.ConvTranspose2d(64, 1, 4, 2, 1, bias=False),
        nn.Tanh()
    )

    def forward(self, input):
        return self.main(input)

# ===== Discriminator =====
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(1, 64, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(64, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Flatten(),
            nn.Linear(128*7*7, 1),
            nn.Sigmoid()
        )

    def forward(self, input):
        return self.main(input)

# ===== Initialize =====
netG = Generator().to(device)
netD = Discriminator().to(device)

criterion = nn.BCELoss()
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))

fixed_noise = torch.randn(64, nz, 1, 1, device=device)

# ===== Training =====
for epoch in range(num_epochs):
    for i, (data, _) in enumerate(dataloader):
        # --- Train Discriminator ---
        netD.zero_grad()

```

```

real = data.to(device)
b_size = real.size(0)
label = torch.full((b_size,), 1., dtype=torch.float, device=device)

output = netD(real).view(-1)
errD_real = criterion(output, label)
errD_real.backward()

noise = torch.randn(b_size, nz, 1, 1, device=device)
fake = netG(noise)
label.fill_(0.)
output = netD(fake.detach()).view(-1)
errD_fake = criterion(output, label)
errD_fake.backward()

optimizerD.step()

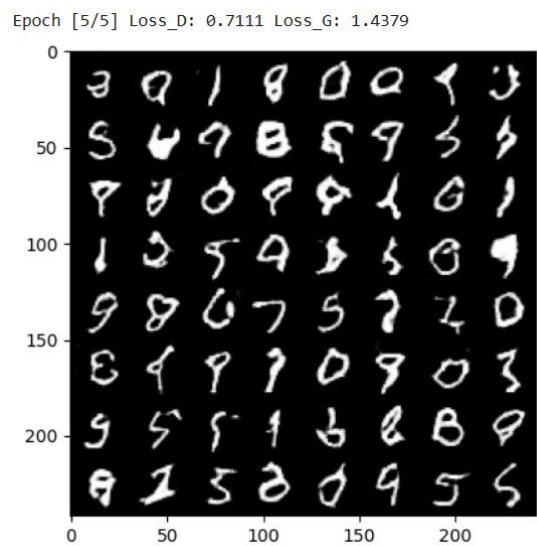
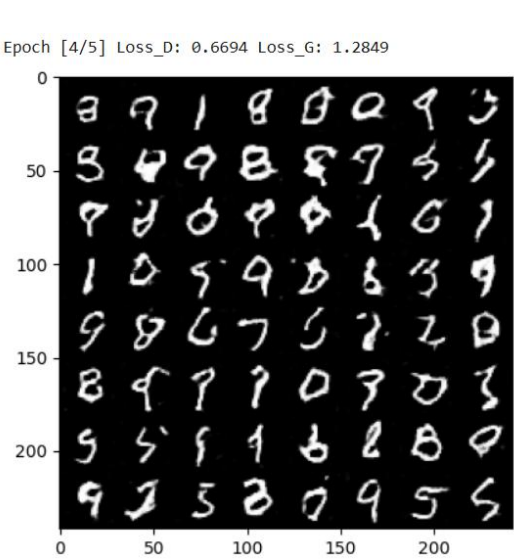
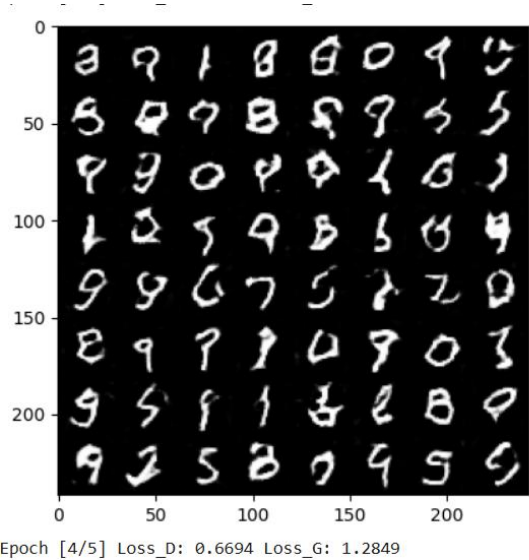
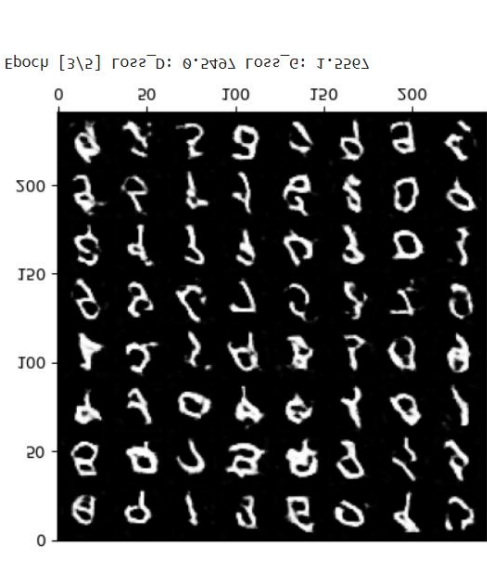
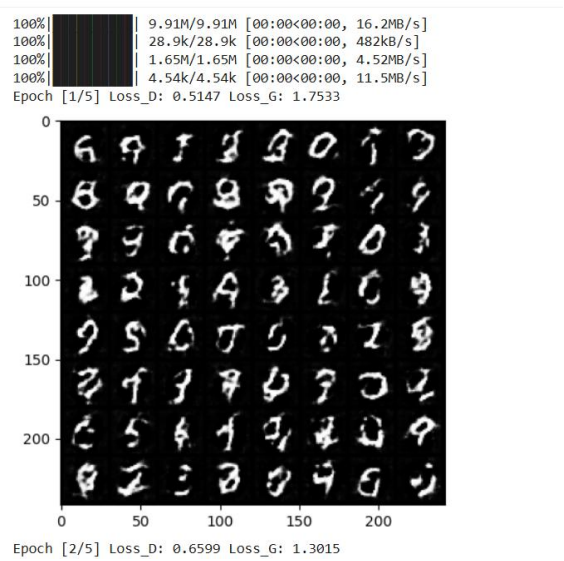
# --- Train Generator ---
netG.zero_grad()
label.fill_(1.)
output = netD(fake).view(-1)
errG = criterion(output, label)
errG.backward()
optimizerG.step()

print(f"Epoch [{epoch+1}/{num_epochs}] Loss_D: {errD_real+errD_fake:.4f} Loss_G: {errG:.4f}")

with torch.no_grad():
    fake = netG(fixed_noise).detach().cpu()
    grid = torchvision.utils.make_grid(fake, padding=2, normalize=True)
    plt.imshow(grid.permute(1, 2, 0))
    plt.show()

```

OUTPUT: -



COE (20)	
RECORD (20)	
VIVA (10)	
TOTAL (50)	

RESULT: -

Thus, the implementation of synthetic image generation using gan is successfully executed.